Project Report: Concurrent Odds-and-Evens Game in Go

Eduard Toni Alexandru

October 24, 2026

1 Analysis

The goal of this project is to design and implement an Odds-and-Evens game in Go, played by N players and organised as a knockout tournament. The requirements identified during the analysis phase are the following.

- **Tournament structure.** The number of players is $N = 2^m$ with $m \geq 1$, so the tournament consists of exactly $m = \log_2 N$ rounds. In round r there are $N/2^r$ matches, which are played concurrently; the winners advance to the next round until the final match determines the champion.
- **Concurrency model.** Interaction must be based on message passing and no shared memory is allowed. Every concurrent entity owns its own state and all coordination and data exchange happens through Go channels.
- **Game rules.** In every match one player takes the role of *odds* and the other the role of *evens*. Both players throw a number of fingers between 0 and 5 simultaneously, i.e. each one chooses without knowing the opponent's choice. If the sum of the two throws is odd the odds player wins, otherwise the evens player wins. Every sum is either odd or even, so a match always has a winner and no tie handling is needed.
- **Fairness.** Since each player has six equally likely values (three even and three odd), the sum is odd in exactly 18 of the 36 equally likely combinations. Both roles therefore win with probability 1/2, and it does not matter which player of a pair is assigned which role. (With throws restricted to 1–5 the odds role would win only with probability 12/25.)
- **Input validation.** The program must accept only integers $N \geq 2$ that are a power of 2, rejecting any other input without getting stuck.

2 Design

The system follows a centralised design with three kinds of active entities: one coordinator, one referee per match and one long-lived goroutine per player.

2.1 Active entities

1. **Main** (`main`): reads and validates N and starts the coordinator.
2. **Coordinator** (`Coord`): creates the N player goroutines, pairs the remaining players at the beginning of each round, starts one referee per match, collects the results and builds the list of players for the next round. At the end it notifies the champion and waits for all players to terminate.

3. **Referees** (`playMatch`): one goroutine per match. A referee asks both players for their throw, applies the parity rule, tells each player whether it won or lost, and reports the winner to the coordinator.

4. **Players** (`Player`): one goroutine per player, alive from the start of the tournament until the player is eliminated (or, for the champion, until the end). A player repeatedly waits for a play request, answers with its throw, and receives the outcome of the match.

Other goroutines refer to a player only through a `PlayerInfo` value containing its identifier and its two input channels (`play` and `result`). Passing a `PlayerInfo` around copies channel references, not state: the only way to interact with a player is to send it a message.

2.2 Message protocol

Table 1 lists all the messages exchanged in the system.

Message	From → To	Channel	Meaning
request (`chan int`)	referee → player	`play`	answer on this channel
throw (`int`)	player → referee	reply channel	the player's throw
outcome (`bool`)	referee → player	`result`	won (stay in) or lost (exit)
`MatchResult`	referee → coordinator	`resultChan`	index of the match and its winner
close of `play`	coordinator → champion	`play`	the tournament is over
exit notification	player → coordinator	`exited`	the player goroutine has finished

Table 1: Messages exchanged between the active entities.

The play request is itself a channel: the referee creates a new channel for each player in each match (`moveChan1`, `moveChan2`) and sends it on the player's `play` channel, and the player answers with its throw on that channel. A player does not need to know its role: the referee knows that the first player of the pair is odds and the second is evens. Because each player answers on its own private channel, a player can never observe the opponent's throw before committing to its own: the *simultaneous throw* required by the rules is guaranteed by the communication structure, not by timing.

Figure 1 shows the messages of one match.

2.3 Player life cycle

A player goroutine alternates between waiting for a match and waiting for its outcome (Figure 2). It terminates in one of two ways: it loses a match, or the coordinator closes its `play` channel because it is the champion. In both cases it sends its identifier on the `exited` channel before terminating.

2.4 Absence of shared memory

- The list of players and the list of winners of the current round are local variables of the coordinator; the coordinator is the only goroutine that reads or writes them.
- Referees receive the two player handles by value and only use them to send messages.
- Throws are generated with `rand.IntN` from `math/rand/v2`, which is safe for concurrent use; the program itself defines no variable that two goroutines access.
- Results reach the coordinator only through the `resultChan` channel. The match index carried by each `MatchResult` lets the coordinator keep the bracket order even though matches finish in any order.

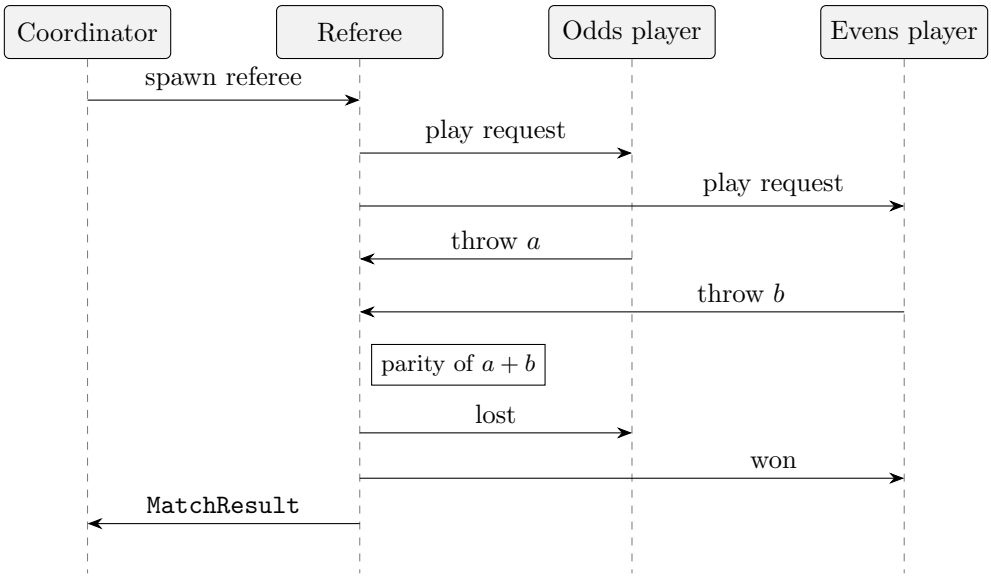


Figure 1: Messages of a single match (here the evens player wins). The two throws travel on separate private channels, so neither player can see the other's throw.

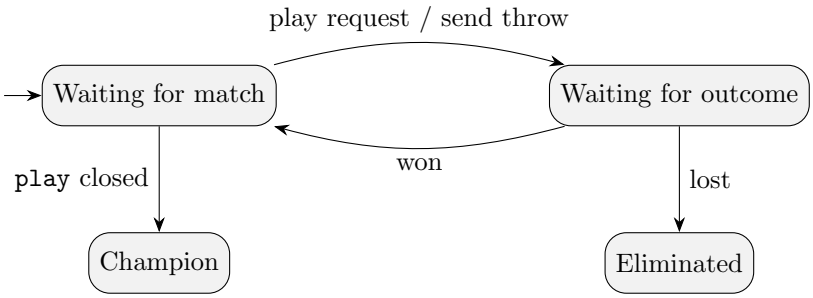


Figure 2: States of a player goroutine. Both final states send an exit notification.

Log lines are printed to standard output by several goroutines. This is output, not program state: no goroutine reads what another one printed.

2.5 Termination and absence of deadlock

- A player is always blocked either on its `play` channel or on its `result` channel, and receives from them in strict alternation. The referee sends on these channels in the same order (request, then outcome), so every send finds its receiver.
- A referee waits for the throws on two channels that only its own two players use, and each player is involved in at most one match at a time, so the two throws always arrive.
- The `resultChan` channel of a round has capacity equal to the number of matches, and the `exited` channel has capacity N. Referees and eliminated players can therefore always deliver their last message and terminate, even before the coordinator reads it.
- After the final, the coordinator closes the champion's `play` channel and receives N exit notifications. When `Coord` prints the winner, every player goroutine has finished and every referee has delivered its result.

3 Implementation and Complexity Analysis

3.1 Pseudocode

Listing 1: Structure of the implementation.

```
function Coord(nPlayers)
    exited = channel of capacity nPlayers
    players = nPlayers PlayerInfo, each with channels play and result
    for each p in players do spawn Player(p, exited)

    roundNum = 1
    while length(players) > 1 do
        nMatches = length(players) / 2
        resultChan = channel of capacity nMatches
        for i = 0 to length(players) - 1 step 2 do
            // first player of the pair plays odds, the second evens
            spawn playMatch(i/2, players[i], players[i+1], roundNum, resultChan)
        nextRoundPlayers = array of size nMatches
        repeat nMatches times
            res = receive from resultChan
            nextRoundPlayers[res.matchIdx] = res.winner
        players = nextRoundPlayers
        roundNum = roundNum + 1

    close(players[0].play)             // tell the champion it is over
    repeat nPlayers times receive from exited
    print "winner is players[0]"

function Player(p, exited)
    for each reply received on p.play do        // loop ends when p.play is closed
        send random value in 0..5 to reply
        if receive from p.result is false then  // lost
            send p.id to exited
            return
    send p.id to exited                         // champion

function playMatch(matchIdx, p1, p2, roundNum, resultChan)
    moveChan1, moveChan2 = new channels
    send moveChan1 to p1.play
    send moveChan2 to p2.play
    m1 = receive from moveChan1
    m2 = receive from moveChan2
    if (m1 + m2) is odd then winner, loser = p1, p2    // odds wins
                        else winner, loser = p2, p1    // evens wins
    send false to loser.result
    send true to winner.result
    send MatchResult(matchIdx, winner) to resultChan
```

3.2 Message complexity

Number of matches. Every match eliminates exactly one player, and all players except the champion are eliminated, so

$$\sum_{r=1}^{\log_2 N} \frac{N}{2^r} = \frac{N}{2} + \frac{N}{4} + \cdots + 1 = N - 1.$$

Messages per match. Every match exchanges exactly 7 messages: 2 play requests, 2 throws, 2 outcome notifications and 1 result sent to the coordinator. Since the parity rule always produces a winner, there are no replays and the number is the same for every match; it is not an expected value.

Shutdown. One close of the champion's `play` channel and N exit notifications.

Total.

$$\text{Messages} = 7(N-1) + (N+1) = 8N - 6 = O(N).$$

N	Rounds	Matches	Match messages	Total messages
2	1	1	7	10
4	2	3	21	26
8	3	7	49	58
16	4	15	105	122
1024	10	1023	7161	8186

Table 2: Exact message counts for some values of N.

3.3 Concurrency and goroutines

- The tournament takes $\log_2 N$ rounds. Rounds are sequential, because a round can only start when its players are known, while all matches of a round run concurrently.
- The system creates N player goroutines and $N-1$ referee goroutines in total. At most $N + N/2$ goroutines are alive at the same time (all players plus the referees of the first round), and the number of live players halves after every round.

3.4 Design alternative: short-lived player goroutines

A simpler design starts a fresh goroutine every time a throw is needed, and represents a player only by its identifier, stored in the coordinator's list. In that design each throw goroutine sends a single message and terminates, so a match needs only 3 messages (2 throws and the result). However, the players are no longer independent processes: they have no life cycle, are never told the outcome of their match, and their identity is data owned by the coordinator. The chosen design costs 4 extra messages per match, but models each player as a process that takes part in the whole tournament and interacts with the rest of the system only through messages. The asymptotic cost is $O(N)$ in both cases.

4 Execution

The program asks for the number of players and rejects non-numeric input, values smaller than 2 and values that are not a power of 2. It was run with the Go race detector (`go run -race .`) for several values of N; no data race was reported, which is consistent with the absence of shared memory.

Sample run

```
--- Start Round 1 (Players left 4) ---
[Player 3] throws 5
[Player 4] throws 5
[Round 1, Match 2] Player 3 (odds) 5 + Player 4 (evens) 5 = 10 -> Player 4 wins
```

```
[Player 2] throws 5
[Player 3] eliminated
[Player 1] throws 1
[Round 1, Match 1] Player 1 (odds) 1 + Player 2 (evens) 5 = 6 -> Player 2 wins
[Player 1] eliminated

--- Start Round 2 (Players left 2) ---
[Player 4] throws 5
[Player 2] throws 0
[Round 2, Match 1] Player 2 (odds) 0 + Player 4 (evens) 5 = 5 -> Player 2 wins
[Player 4] eliminated
[Player 2] I am the champion!

 !!! THE WINNER OF THE CHAMPIONSHIP IS 2!
```

The lines of the two first-round matches are interleaved, which shows that the matches run concurrently, while the final starts only after both have finished.